

ESCOLA SESI/SENAI DE SALTO
CENTRO EDUCACIONAL 125
ENSINO MÉDIO INTEGRADO AO TÉCNICO

EDUARDO HENRIQUE DE JESUS CHAVES
HELOISA GABRIELLY PAIXÃO
PEDRO DA SILVA ANDRES JIMENEZ

PESQUISA - UPLOAD DE ARQUIVOS NA API REST

SALTO
2026

EDUARDO HENRIQUE DE JESUS CHAVES

HELOISA GABRIELLY PAIXÃO

PEDRO DA SILVA ANDRES JIMENEZ

PESQUISA - UPLOAD DE ARQUIVOS NA API REST

Trabalho apresentado à disciplina Programação Back-End do Ensino Médio e Técnico da Escola SESI/SENAI de Salto, como avaliação parcial do 1º semestre/etapa do 3º ano B.

Professor(a): Celso Giusti, Daniel Manoel Filho, Marlon Fanger Rodrigues.

SALTO

2026

1. Introdução	4
2. Por que o upload de arquivos é necessário?	4
3. A Escolha da Solução	4
3.1 Por que o Multer?	4
3.2 Arquivos e o banco de dados	5
4. Fluxo dos dados da requisição	5
5. Adaptação da API	5
6. Camada de segurança	9
6.1 Salvamento dos arquivos	10
7. Conclusão	10
8. Referências bibliográficas	11

1. Introdução

Ao decorrer das aulas de Programação Back-End, foi notado a necessidade de conter um atributo na entidade “Produto” que suportasse o carregamento de arquivos, como imagens de diferentes extensões ou até mesmo documentos externos que facilitariam a organização final da aplicação “Sabor Digital”. Para que esse objetivo seja alcançado, serão feitas pesquisas sobre as bibliotecas, rotas e modelos de desenvolvimento necessários e recomendados.

Foi usada a estrutura de padrão de desenvolvimento MVC, onde as responsabilidades dentro da API são divididas e garantem que suas tarefas sejam feitas focadas em seu objetivo. Além disso, é importante ressaltar o uso inicial do ambiente de desenvolvimento Node.js, as bibliotecas Express e Dotenv e o banco de dados relacional do MySQL2.

2. Por que o upload de arquivos é necessário?

O desenvolvimento de aplicações web modernas frequentemente exige a manipulação de arquivos de mídia, como imagens para perfis de usuários ou catálogos de produtos. No ecossistema Node.js, o recebimento desses arquivos através de requisições HTTP introduz uma complexidade técnica: os dados textuais comuns (JSON) trafegam de forma diferente dos dados binários (arquivos).

Ao trabalhar com o framework Express, o servidor nativamente não possui a capacidade de interpretar requisições do tipo *multipart/form-data*, o padrão utilizado pelos navegadores e ferramentas de teste (como o Postman) para o envio de arquivos. Portanto, para o desenvolvimento desta funcionalidade, foi necessária a integração de uma ferramenta especialista para realizar a interceptação, a leitura binária e o armazenamento físico desses dados no servidor.

3. A Escolha da Solução

Para suprir a limitação do Express no tratamento de fluxos de mídia, a comunidade Node.js adota amplamente a biblioteca Multer. Onde, segundo pesquisas em diversos sites de desenvolvimento, é o que o mercado mais usa e o que é mais flexível em relação ao desenvolvimento e estabelecimento de regras de negócio complexas (como tamanho do arquivo ou nome do mesmo).

3.1 Por que o Multer?

- **Eficiência no tratamento:** O Multer processa o arquivo em partes (chunks), evitando que o servidor estoure o limite de memória RAM ao receber arquivos grandes.
- **Separação de escopo:** Ele injeta o arquivo no objeto *req.file* e os textos comuns no *req.body*, permitindo que a API trate os dados separadamente de forma organizada, facilitando o acesso do desenvolvedor ao banco.

- **Configuração de Armazenamento:** Através do método *diskStorage*, ele oferece controle sobre o destino do arquivo e a renomeação automática para evitar a sobrescrita de arquivos com nomes iguais.

3.2 Arquivos e o banco de dados

Uma decisão crucial de arquitetura tomada neste projeto foi armazenar o arquivo físico no disco rígido (pasta */uploads*) e apenas a string do seu caminho estruturado no banco de dados MySQL.

Nota: Salvar arquivos binários diretamente no banco de dados (usando campos do tipo BLOB) degrada severamente a performance das consultas ao banco de dados SQL, aumenta o tamanho dos backups e aumenta o custo de latência (tempo de atraso entre o envio de uma informação e o recebimento da resposta correspondente) da aplicação. Armazenar referências em texto (URLs) resolve este problema.

4. Fluxo dos dados da requisição

Para compreender como a solução opera dentro do sistema, é fundamental mapear o ciclo de vida de uma requisição de upload.

1. **Cliente:** Envia uma requisição POST com os campos de texto e o arquivo binário encapsulados em um formulário *form-data*.
2. **Rotas (Express):** O endpoint intercepta a chamada e aciona o Multer como um *middleware*.
3. **Multer:** Extrai o binário, salva o arquivo na pasta */uploads* com um nome único baseado em timestamp (*Date.now()*) e anexa os metadados (dados sobre outros dados) em *req.file*.
4. **Controller:** Recebe o controle da requisição contendo tanto o arquivo processado quanto os textos, envelopando-os em um único pacote de transferência de dados.
5. **Service:** Executa as validações de segurança e regras de negócio sobre a imagem e os textos que foram enviados pelo usuário.
6. **Repository:** Interage com o banco de dados via MySQL2, persistindo a string do caminho da imagem no registro correspondente que foi pré-definido pelo desenvolvedor direito do MySQL Workbench.

5. Adaptação da API

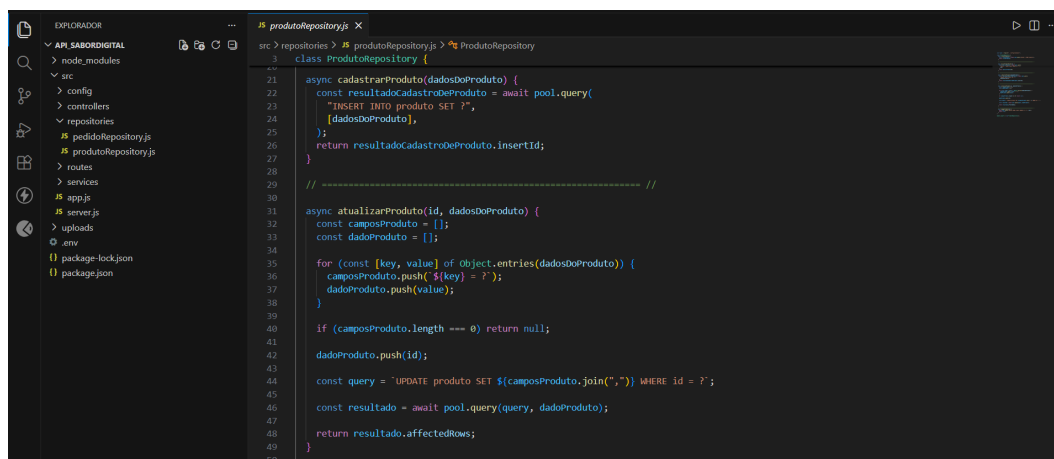
A implementação do Multer exigiu uma refatoração para que a API não perdesse a separação de responsabilidades que foram definidas pelo padrão MVC (Model-View-Controller). A grande dificuldade centralizada estava em onde validar os erros da imagem sem poluir as rotas ou o controlador, pois as documentações

apontavam para a validação de erros diretamente na definição de rotas, algo que não coincidia com o modelo de referência.

Além disso, foi necessária a execução de um comando ALTER TABLE para introduzir o atributo *foto_produto* na tabela de produtos. Este campo foi tipificado como VARCHAR com comprimento variável suficiente para suportar strings de caminhos locais absolutos de serviços de armazenamento em nuvem.

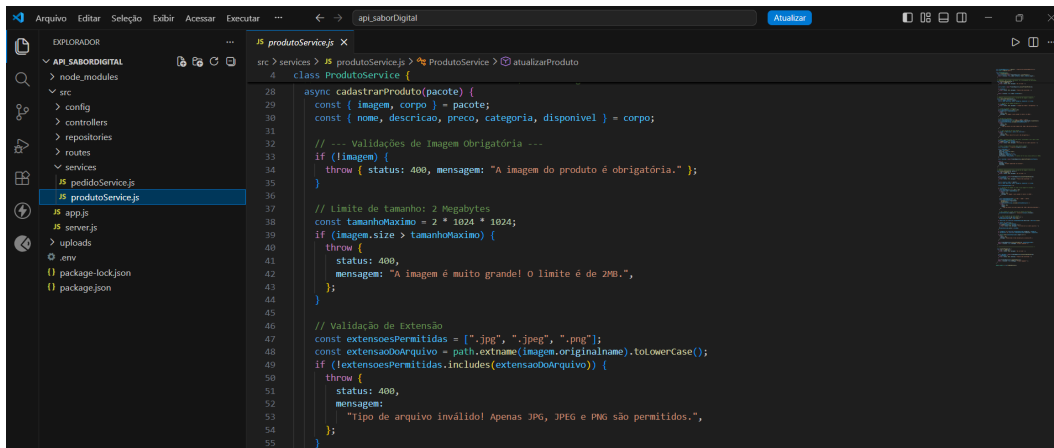
A arquitetura foi adaptada dividindo as responsabilidades da seguinte forma:

A Repository exerce o papel de interface direta com o sistema de gerenciamento de banco de dados (SGBD) MySQL. Esta camada limita-se a receber o objeto estruturado pela Service e mapear suas propriedades diretamente como colunas da tabela, persistindo a referência textual da imagem de maneira automatizada e transparente. Em nossa *repository*, não houveram maiores mudanças por se tratar de um arquivo que faz conexão direta com o banco de dados e colher as informações das outras camadas de serviço.



A Service concentra as validações de segurança do sistema. Esta camada é responsável por avaliar o objeto recebido do Controller, aplicando os critérios predefinidos, como a verificação de presença do arquivo, a restrição volumétrica de 2MB e o filtro de extensões permitidas (.jpg, .jpeg, .png). Caso algo seja violado, a classe interrompe o fluxo. Dentro dela foram feitas as seguintes alterações:

- Validações obrigatórias de imagem, como o formato, o tamanho e sua obrigatoriedade;
- Criação de uma string url para salvar essa imagem dentro do banco de dados sem danificar os arquivos.
- Uso do path é um componente nativo do Node.js e a função principal dele é gerenciar e formatar caminhos de arquivos e pastas no sistema operacional, extraíndo nomes e extensões do arquivo original para usá-lo futuramente num caminho válido.

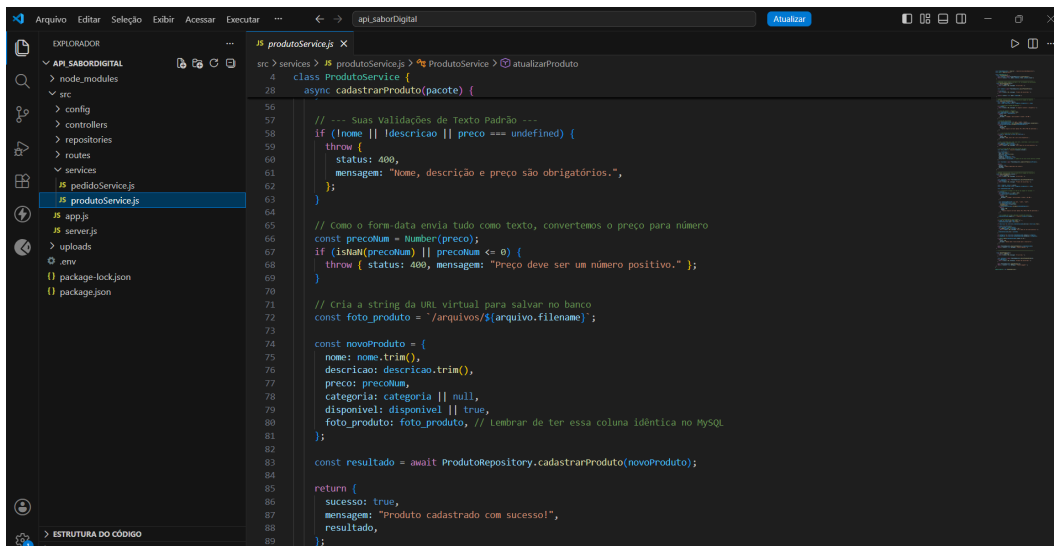


Arquivo Editar Seleção Exibir Acessar Executar ... api_saborDigital Atualizar

EXPLORADOR

- API_SABORDIGITAL
 - node_modules
 - src
 - config
 - controllers
 - repositories
 - routes
 - services
 - pedidoService.js
 - produtoService.js**
 - app.js
 - server.js
 - uploads
 - env
 - package-lock.json
 - package.json

```
src > services > JS produtoService.js > ProdutoService > atualizarProduto
4 class ProdutoService {
28
29   async cadastrarProduto(pacote) {
30     const { imagem, corpo } = pacote;
31     const { nome, descricao, preco, categoria, disponivel } = corpo;
32
33     // --- Validações de Imagem Obrigatória ---
34     if (!imagem) {
35       throw { status: 400, mensagem: "A imagem do produto é obrigatória." };
36     }
37
38     // Limite de tamanho: 2 Megabytes
39     const tamanhoMaximo = 2 * 1024 * 1024;
40     if (imagem.size > tamanhoMaximo) {
41       throw {
42         status: 400,
43         mensagem: "A imagem é muito grande! O limite é de 2MB.",
44       };
45     }
46
47     // Validação de Extensão
48     const extensoesPermitidas = [".jpg", ".jpeg", ".png"];
49     const extensaoDoArquivo = path.extname(imagem.originalname).toLowerCase();
50     if (!extensoesPermitidas.includes(extensaoDoArquivo)) {
51       throw {
52         status: 400,
53         mensagem: "Tipo de arquivo inválido! Apenas JPG, JPEG e PNG são permitidos.",
54       };
55     }
56   }
57 }
```

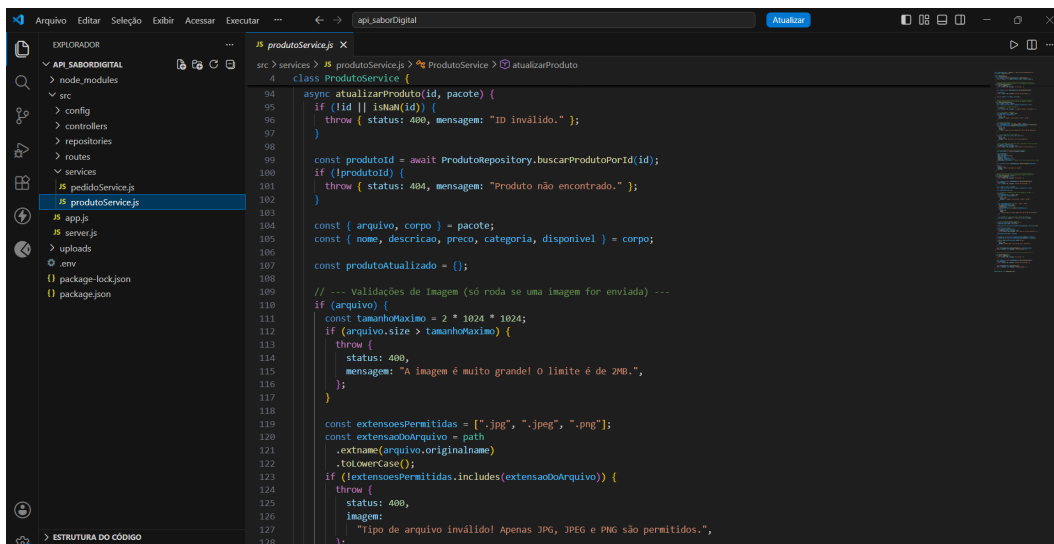


Arquivo Editar Seleção Exibir Acessar Executar ... api_saborDigital Atualizar

EXPLORADOR

- API_SABORDIGITAL
 - node_modules
 - src
 - config
 - controllers
 - repositories
 - routes
 - services
 - pedidoService.js
 - produtoService.js**
 - app.js
 - server.js
 - uploads
 - env
 - package-lock.json
 - package.json

```
src > services > JS produtoService.js > ProdutoService > atualizarProduto
4 class ProdutoService {
28   async cadastrarProduto(pacote) {
29
30     // --- Suas Validações de Texto Padrão ---
31     if (!nome || !descricao || !preco || !categoria || !disponivel) {
32       throw {
33         status: 400,
34         mensagem: "Nome, descrição e preço são obrigatórios.",
35       };
36     }
37
38     // Como o form-data envia tudo como texto, convertemos o preco para número
39     const precoNum = Number(preco);
40     if (isNaN(precoNum) || precoNum <= 0) {
41       throw { status: 400, mensagem: "Preço deve ser um número positivo." };
42     }
43
44     // Cria a string da URL virtual para salvar no banco
45     const foto_produto = "/arquivos/${arquivo.filename}";
46
47     const novoProduto = {
48       nome: nome.trim(),
49       descricao: descricao.trim(),
50       preco: precoNum,
51       categoria: categoria || null,
52       disponivel: disponivel || true,
53       foto_produto: foto_produto, // Lembrear de ter essa coluna idêntica no MySQL
54     };
55
56     const resultado = await ProdutoRepository.cadastrarProduto(novoProduto);
57
58     return {
59       sucesso: true,
60       mensagem: "Produto cadastrado com sucesso!",
61       resultado,
62     };
63   }
64 }
```

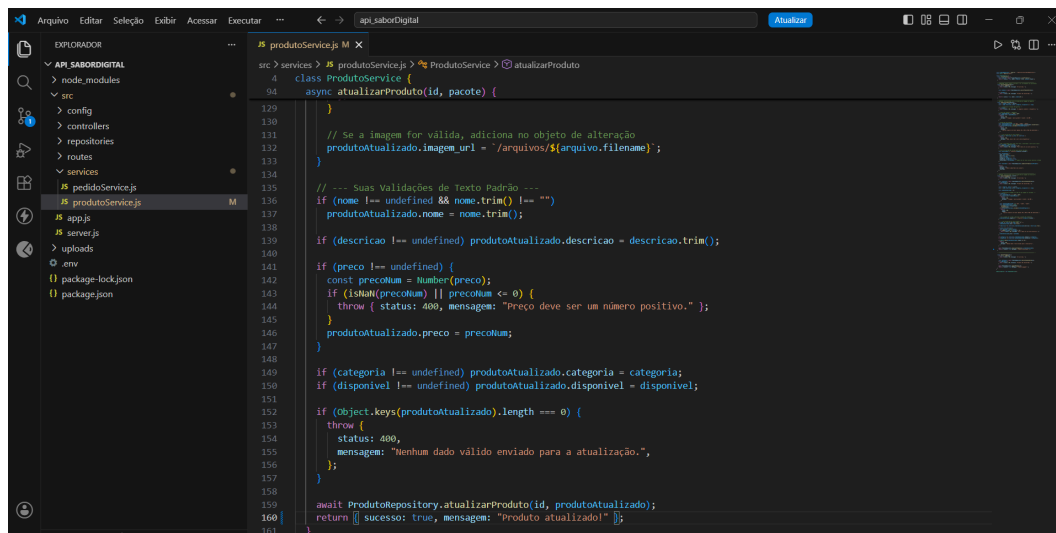


Arquivo Editar Seleção Exibir Acessar Executar ... api_saborDigital Atualizar

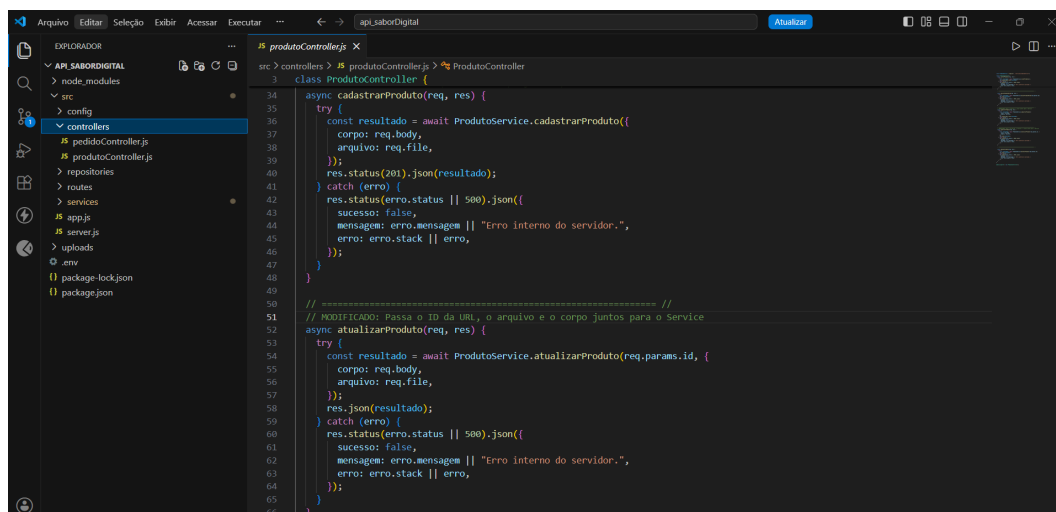
EXPLORADOR

- API_SABORDIGITAL
 - node_modules
 - src
 - config
 - controllers
 - repositories
 - routes
 - services
 - pedidoService.js
 - produtoService.js**
 - app.js
 - server.js
 - uploads
 - env
 - package-lock.json
 - package.json

```
src > services > JS produtoService.js > ProdutoService > atualizarProduto
4 class ProdutoService {
94   async atualizarProduto(id, pacote) {
95     if (!id || !isId(id)) {
96       throw { status: 400, mensagem: "ID inválido." };
97     }
98
99     const produtoId = await ProdutoRepository.buscarProdutoPorId(id);
100     if (!produtoId) {
101       throw { status: 404, mensagem: "Produto não encontrado." };
102     }
103
104     const { arquivo, corpo } = pacote;
105     const { nome, descricao, preco, categoria, disponivel } = corpo;
106
107     const produtoAtualizado = {};
108
109     // --- Validações de Imagem (só roda se uma imagem for enviada) ---
110     if (arquivo) {
111       const tamanhoMaximo = 2 * 1024 * 1024;
112       if (arquivo.size > tamanhoMaximo) {
113         throw {
114           status: 400,
115           mensagem: "A imagem é muito grande! O limite é de 2MB.",
116         };
117       }
118
119       const extensoesPermitidas = [".jpg", ".jpeg", ".png"];
120       const extensaoDoArquivo = path
121         .extname(arquivo.originalname)
122         .toLowerCase();
123       if (!extensoesPermitidas.includes(extensaoDoArquivo)) {
124         throw {
125           status: 400,
126           mensagem: "Tipo de arquivo inválido! Apenas JPG, JPEG e PNG são permitidos.",
127         };
128       }
129     }
130   }
131 }
```



A Controller atua estritamente como um intermediário de entrada e saída, agindo conforme as regras de negócio. Sua única função nesta implementação é interceptar a requisição HTTP e unificar os dados textuais do req.body com os metadados do arquivo binário processados pelo Multer (req.file).

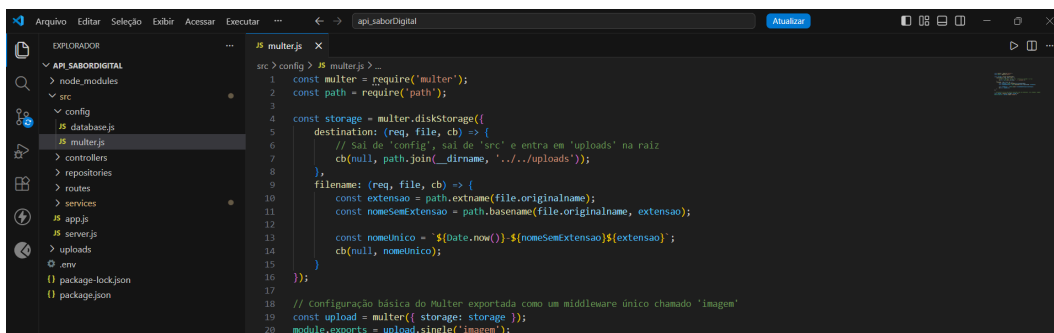


O Multer foi isolado para atuar como um operador de disco (orquestrador físico). Ele apenas recebe e posiciona o arquivo na pasta física correta e não sendo responsável por tomar decisões de negócio que foram definidas antecipadamente. Para a criação desse arquivo foi necessário os seguintes componentes:

- Cópia do método de armazenamento no disco físico (diskStorage). Este método permite configurar as diretrizes de salvamento do arquivo em duas funções operacionais: destination e filename;
- Função callback encarregada de decidir o diretório físico do servidor onde o arquivo processado será colocado;
- Retorno do Multer (*callback*) com o primeiro parâmetro (*null*) que diz a ausência de erros na operação e o segundo parâmetro que usa o

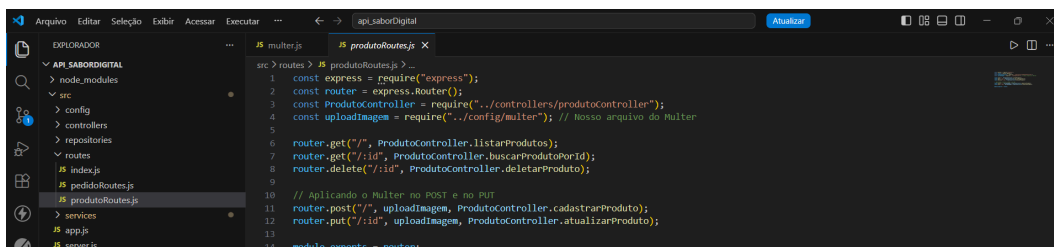
método `path.join` combinando o diretório atual (`__dirname`) com referências relativas (`../../uploads`) para achar o caminho exato da pasta destino.

- Uso do método `path.extname` para extrair o sufixo de extensão (ex: `.png`, `.jpg`) do arquivo original;
- Uso do método `path.basename` passando a extensão como argumento de exclusão, isolando o nome do arquivo, expurgando a sua extensão original.
- Geração de um nome único para o arquivo em seu destino e retorno da função de nomenclatura, com o nome padronizado final para o arquivo gravado.
- Criação do construtor Multer, injetando as propriedades e regras de armazenamento configuradas no objeto storage.
- Exportação do módulo configurado no formato de um middleware Express exclusivo. O método `.single('imagem')` restringe o recebimento de apenas um arquivo por requisição e define a chave identificadora no corpo do formulário como 'imagem'.



```
src > config > JS multer.js > ...
1  const multer = require('multer');
2  const path = require('path');
3
4  const storage = multer.diskStorage({
5    destination: (req, file, cb) => {
6      // Sai de 'config', sai de 'src' e entra em 'uploads' na raiz
7      cb(null, path.join(__dirname, '../../uploads'));
8    },
9    filename: (req, file, cb) => {
10     const extensao = path.extname(file.originalname);
11     const nomeSemExtensao = path.basename(file.originalname, extensao);
12
13     const nomeUnico = `${Date.now()}-${nomeSemExtensao}${extensao}`;
14     cb(null, nomeUnico);
15   });
16
17 // Configuração básica do Multer exportada como um middleware único chamado 'imagem'
18 const upload = multer({ storage: storage });
19 module.exports = upload.single('imagem');
```

As rotas apenas executam o Multer. Para manter o Controller e o Service limpos e independentes de estruturas do Express, o Controller foi adaptado para unificar os dados textuais (`req.body`) com o arquivo binário (`req.file`) em um único objeto de transporte que será salvo no banco de dados.



```
src > routes > JS produtoRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const ProdutoController = require('../controllers/produtoController');
4  const uploadImage = require('../config/multer'); // Nosso arquivo do Multer
5
6  router.get('/', ProdutoController.listarProdutos);
7  router.get('/:id', ProdutoController.buscarProdutoPorId);
8  router.delete('/:id', ProdutoController.deletarProduto);
9
10 // Aplicando o Multer no POST e no PUT
11 router.post('/', uploadImage, ProdutoController.cadastrarProduto);
12 router.put('/:id', uploadImage, ProdutoController.atualizarProduto);
13
14 module.exports = router;
```

6. Camada de segurança

Permitir que usuários façam upload de arquivos sem restrições faz com que vulnerabilidades críticas de segurança (como ataques de negação de serviço por

arquivos gigantes ou execução de scripts maliciosos) sejam mais frequentes. Três validações fundamentais foram implementadas diretamente no código da **Service**:

1. Garante que o fluxo de cadastro não continue caso o campo de arquivo venha vazio, retornando um erro controlado 400 Bad Request.
2. Através do módulo nativo *path*, a API extrai a extensão do arquivo enviado e verifica se ela consta na lista de formatos permitidos (*.jpg*, *.jpeg*, *.png*). Isso impede que o usuário suba arquivos executáveis (*.exe*) ou scripts perigosos disfarçados.
3. O tamanho do arquivo (medido em bytes no atributo *arquivo.size*) é checado contra um teto rígido de 2 Megabytes ($2 \times 1024 \times 1024$ bytes). Arquivos maiores são rejeitados imediatamente para proteger o armazenamento do servidor.

6.1 Salvamento dos arquivos

A pasta física de armazenamento se chama uploads e reside localmente na raiz do servidor. Portanto, quando o banco de dados armazena a string `/arquivos/imagem-123.jpg`, qualquer cliente conseguirá visualizar a imagem fazendo uma requisição direta para a URL pública gerada: `http://localhost:3000/arquivos/imagem-123.jpg`. Isso protege a árvore real de diretórios do sistema contra atitudes de usuários maliciosos.

7. Conclusão

Em resumo, a integração do Multer foi fundamental para fazer a nossa API evoluir. Com ele, o sistema agora é capaz de receber imagens de forma organizada e segura, separando o que é texto do que é arquivo de foto. A grande vantagem desta implementação foi seguir à risca o padrão do projeto:

- O banco de dados **MySQL** não ficou pesado, pois salvamos nele apenas o "link" em texto da foto, deixando o arquivo guardado na pasta correta do computador.
- As **Rotas** e o **Controller** continuaram simples, servindo apenas como pontes para levar os dados adiante.
- O **Service** centralizou todas as regras e bloqueou de forma inteligente fotos muito grandes (maiores que 2MB) ou com formatos errados.

Com isso, conseguimos desenvolver uma aplicação que aceita o cadastro de produtos com foto, mas que continua leve, segura e muito bem organizada.

8. Referências bibliográficas

EXPRESS. **Express/Node.js web application framework**. Versão 4.x. Disponível em: <https://expressjs.com/pt-br/>. Acesso em: 25 maio 2026.

MULTER. **Multer: Node.js middleware for handling multipart/form-data**. Versão 1.4.x. npm, 2026. Disponível em: <https://www.npmjs.com/package/multer>. Acesso em: 25 maio 2026.

NODE.JS. **Path: Node.js v20.x API documentation**. Node.js Foundation, 2026. Disponível em: <https://nodejs.org/api/path.html>. Acesso em: 25 maio 2026.

Parte do conteúdo deste documento foi corrigido por IA com propósito de deixar o conteúdo mais dinâmico, revisar e explicar conteúdos e conceitos novos e complexos.